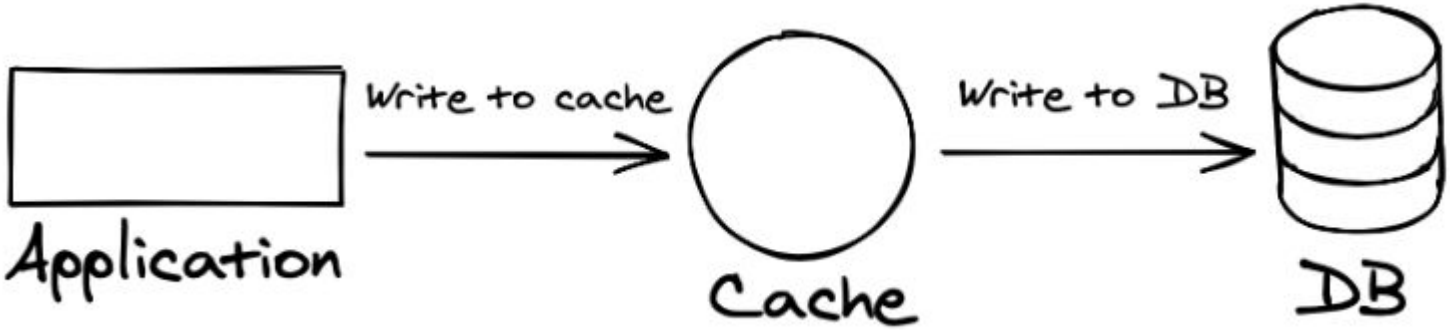
# Lesson 35

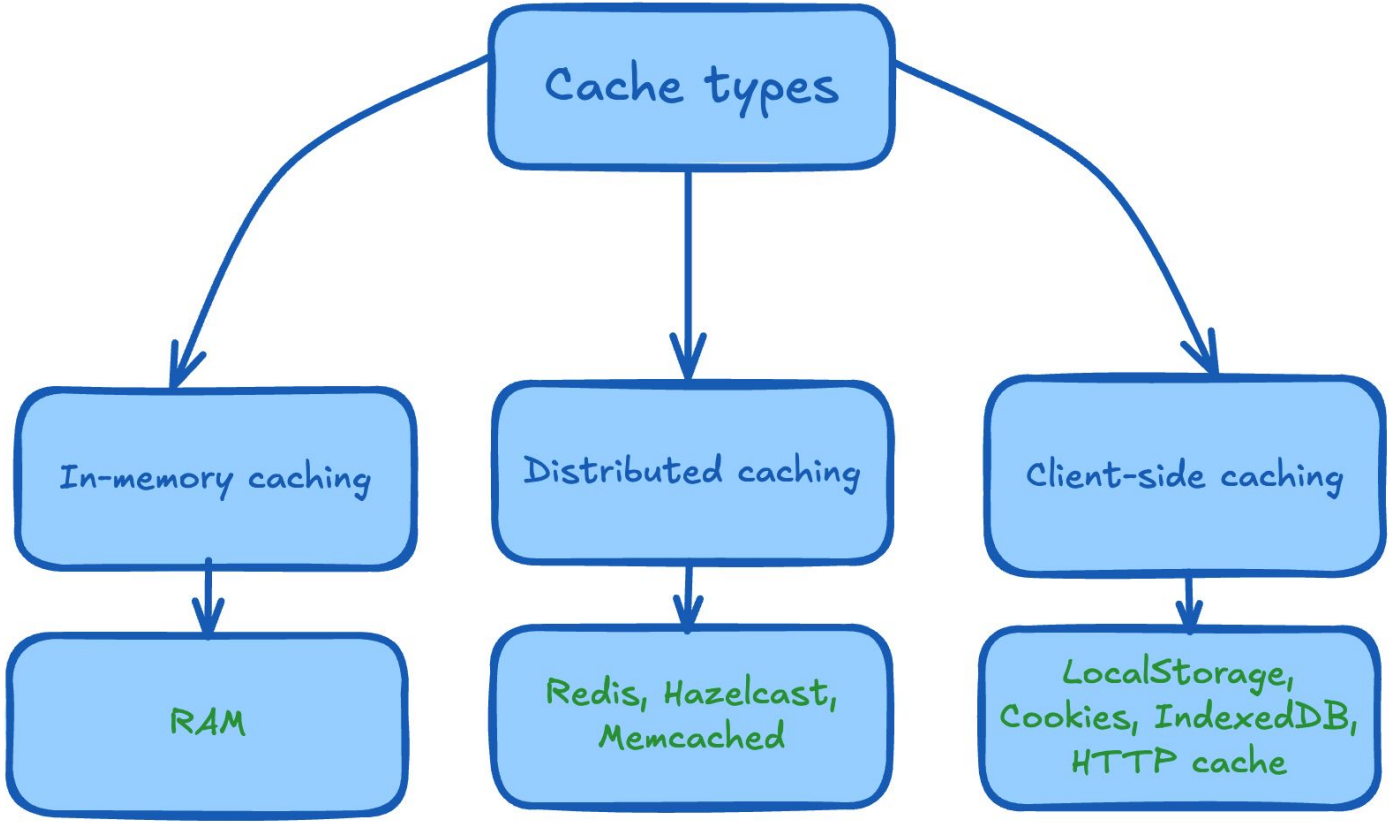
30.04.2026

Cache

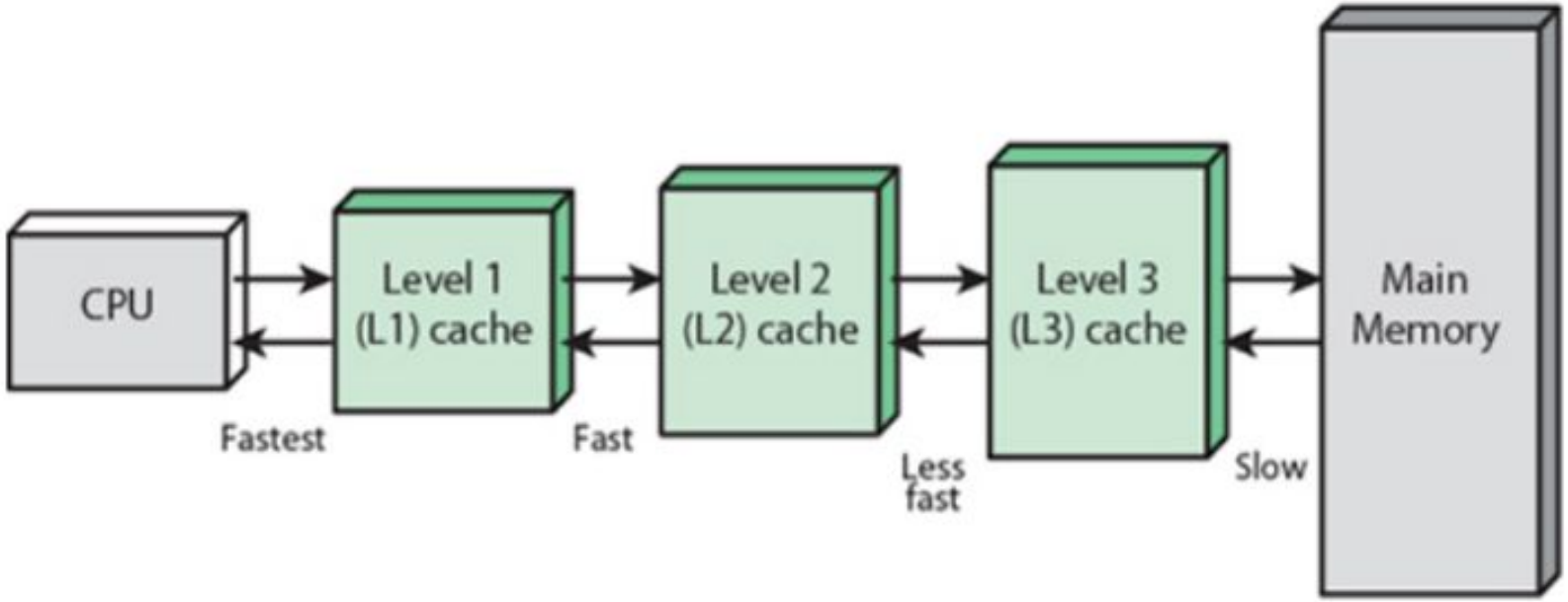
**Що таке кешування?**

У сфері обчислювальної обробки даних кеш – це високошвидкісний рівень зберігання, у якому необхідний набір даних, зазвичай, тимчасового характеру. Доступ до даних на цьому рівні здійснюється значно швидше, ніж до основного місця їхнього зберігання. За допомогою кешування стає можливим ефективне повторне використання раніше отриманих чи обчислених даних.




Cache types
In-memory caching
Distributed caching
Client-side caching
RAM
Redis, Hazelcast, Memcached
LocalStorage, Cookies, IndexedDB, HTTP cache

# Types of Cache Memory

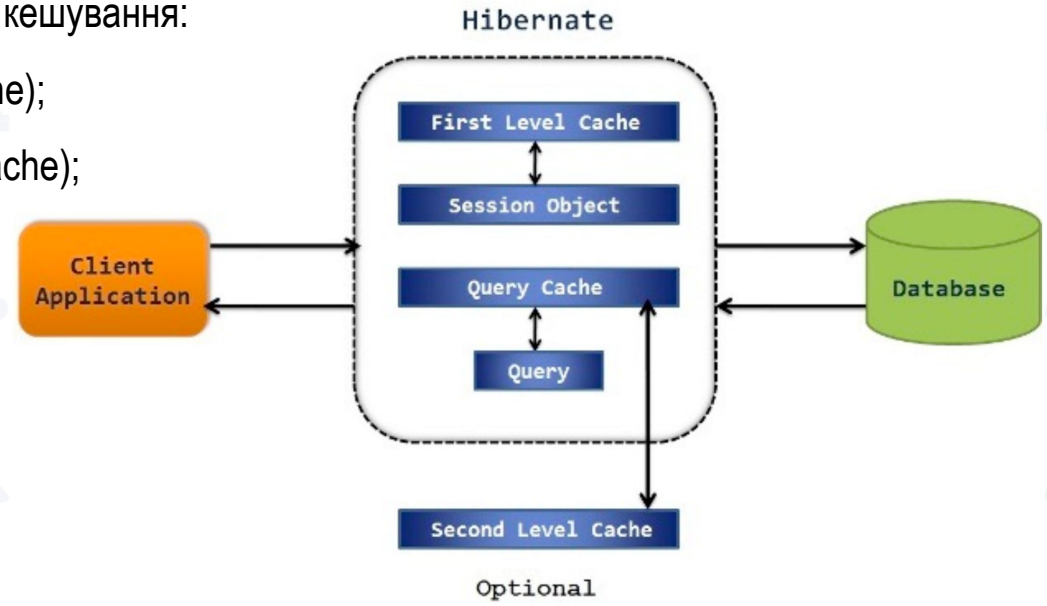


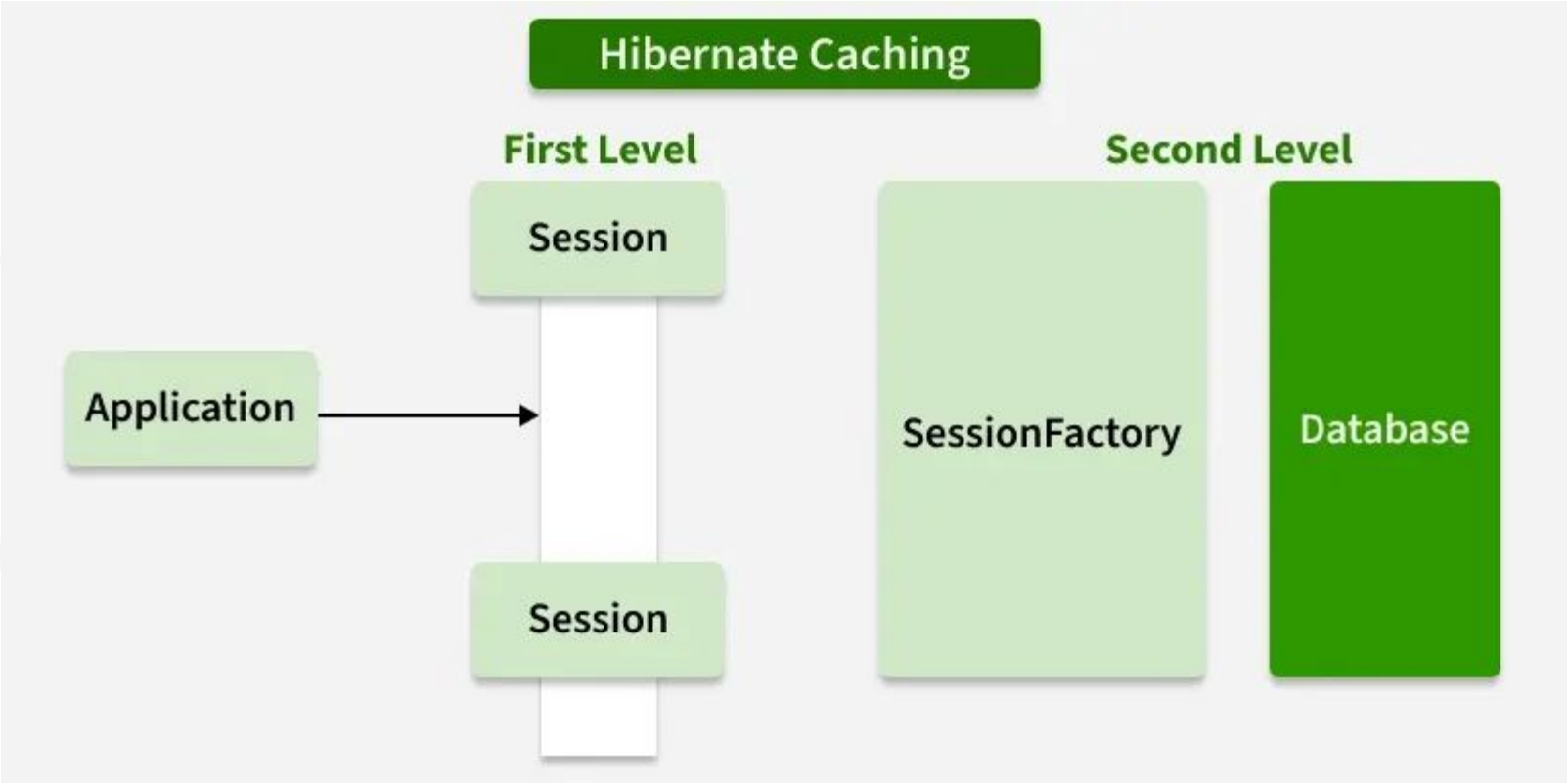
## Hibernate cache

Досить часто в Java додатках з метою зниження навантаження на БД використовують кеш. Небагато людей реально розуміють як працює кеш під капотом, додати просто анотацію не завжди достатньо, потрібно розуміти як працює система.

Насамперед Hibernate cache - це 3 рівні кешування:

- Кеш першого рівня (First-level cache);
- Кеш другого рівня (Second-level cache);
- Кеш запитів (Query cache);




Hibernate Caching
First Level
Second Level
Session
Application
Session
SessionFactory
Database

## Кеш першого рівня

Кеш першого рівня завжди прив'язаний до об'єкта сесії. Hibernate завжди за промовчанням використовує цей кеш і його не можна вимкнути. Давайте відразу розглянемо наступний код:

```
SharedDoc persistedDoc = (SharedDoc) session.load(SharedDoc.class, docId);
System.out.println(persistedDoc.getName());
user1.setDoc(persistedDoc);

persistedDoc = (SharedDoc) session.load(SharedDoc.class, docId);
System.out.println(persistedDoc.getName());
user2.setDoc(persistedDoc);
```

## Кеш запитів

```
Query query = session.createQuery("from SharedDoc doc where doc.name = :name");

SharedDoc persistedDoc = (SharedDoc) query.setParameter("name", "first").uniqueResult();
System.out.println(persistedDoc.getName());
user1.setDoc(persistedDoc);

persistedDoc = (SharedDoc) query.setParameter("name", "first").uniqueResult();
System.out.println(persistedDoc.getName());
user2.setDoc(persistedDoc);
```

Результати таких запитів не зберігаються ні кешем першого, ні другого рівня. Це саме місце, де можна використовувати кеш запитів. Він теж за замовчуванням вимкнено. Для включення потрібно додати наступний рядок до конфігураційного файлу:

```
<property name="hibernate.cache.use_query_cache" value="true"/>
```

```
Query query = session.createQuery("from SharedDoc doc where doc.name = :name");
query.setCacheable(true);
```

## Кеш другого рівня

Якщо кеш першого рівня прив'язаний до об'єкта сесії, кеш другого рівня прив'язаний до об'єкта-фабрики сесій (Session Factory object). Що має на увазі, що видимість цього кеша набагато ширша за кеш першого рівня.

```
Session session = factory.openSession();
SharedDoc doc = (SharedDoc) session.load(SharedDoc.class, 1L);
System.out.println(doc.getName());
session.close();

session = factory.openSession();
doc = (SharedDoc) session.load(SharedDoc.class, 1L);
System.out.println(doc.getName());
session.close();
```

У цьому прикладі буде виконано 2 запити до бази, це пов'язано з тим, що за замовчуванням кеш другого рівня вимкнено. Для включення необхідно додати наступні рядки у конфігураційному файлі

```
<property name="hibernate.cache.provider_class" value="net.sf.ehcache.hibernate.SingletonEhCach
eProvider"/>
//или  в более старых версиях
//<property name="hibernate.cache.provider_class" value="org.hibernate.cache.EhCacheProvider"/>
<property name="hibernate.cache.use_second_level_cache" value="true"/>
```

Хибернейт сам не реализует кеширование как таковое. А лишь предоставляет структуру для его реализации, поэтому подключить можно любую реализацию, которая соответствует спецификации нашего ORM фреймворка. Из популярных реализаций можно выделить следующие:

- EHCache
- OSCache
- SwarmCache
- JBoss TreeCache

Крім того, швидше за все, Вам також знадобиться окремо налаштувати і саму реалізацію кеша. У випадку з EHCache, це потрібно зробити у файлі ehcache.xml. Ну і на завершення ще потрібно вказати самому хибернейту, що кешувати. На щастя, це дуже легко можна зробити за допомогою анотацій, наприклад:

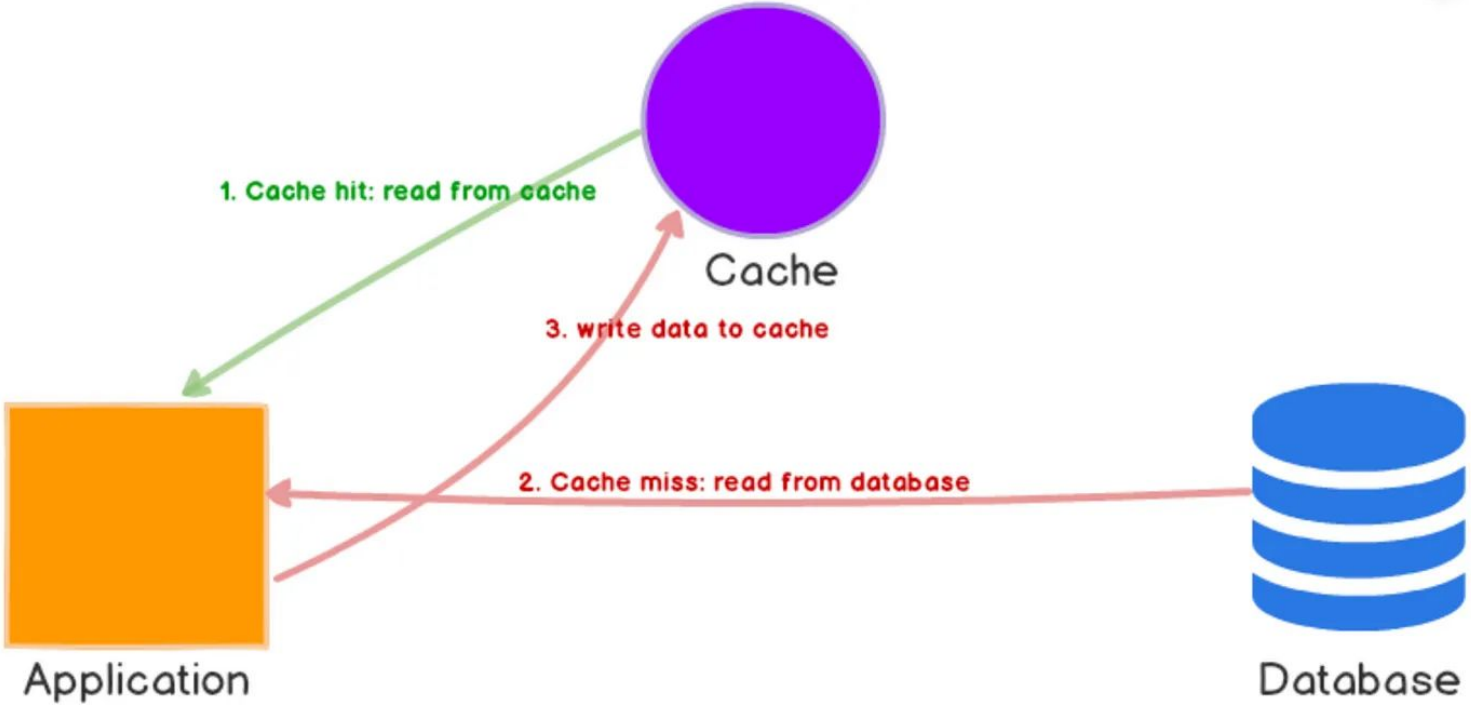
```
@Entity
@Table(name = "shared_doc")
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class SharedDoc{
    private Set<User> users;
}
```

Стратегії кешування визначають поведінки кеша у певних ситуаціях.
Виділяють чотири групи:

- Read-only
- Read-write
- Nonstrict-read-write
- Transactional

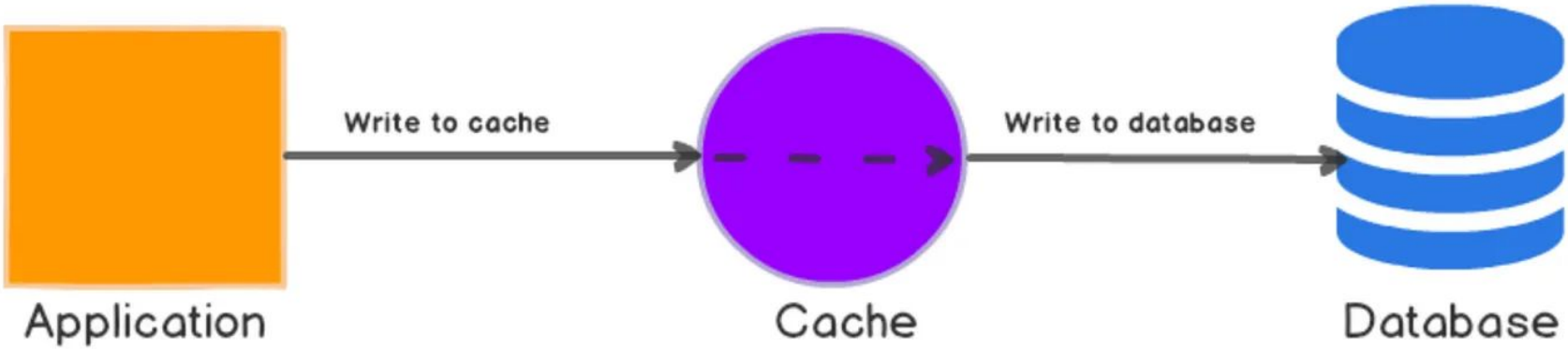
## Cache-Aside

У цій стратегії програма відповідає за керування кеш-пам’яттю. Коли запитуються дані, програма спочатку перевіряє кеш. Якщо даних немає в кеші, вони витягуються з бази даних і зберігаються в кеші для подальшого використання. Ця стратегія є простою та гнучкою, але вона вимагає ретельного керування кеш-пам’яттю, щоб забезпечити її актуальність.



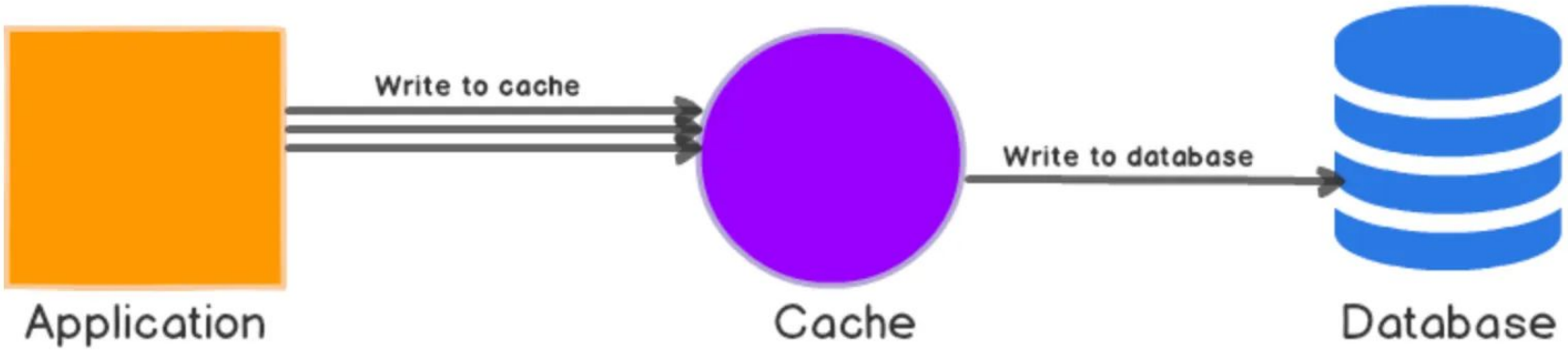
## Write-Through

У цій стратегії дані записуються як у кеш, так і в базу даних одночасно. Коли дані оновлюються, вони записуються в кеш і базу даних одночасно. Це гарантує, що кеш завжди містить актуальні дані, але це може уповільнити операції запису.



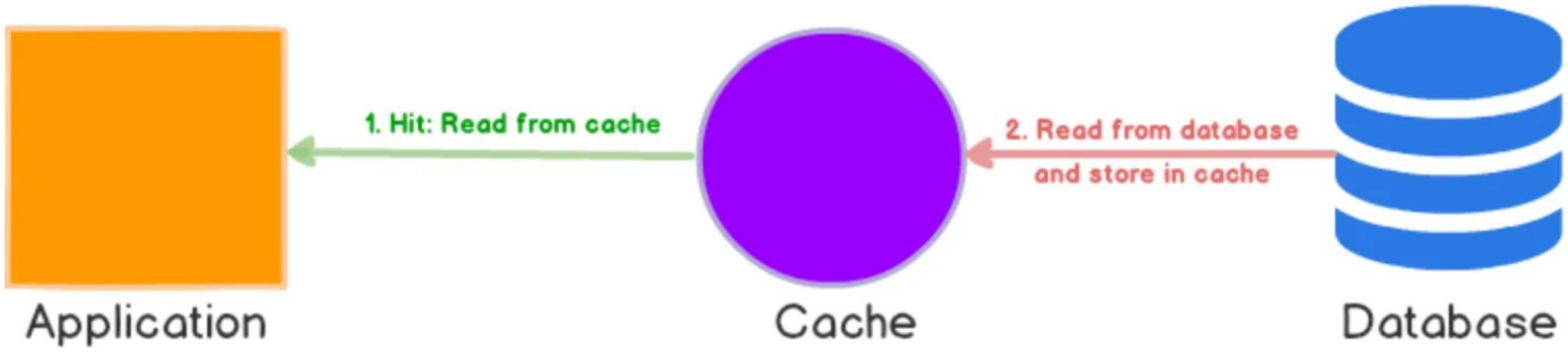
## Write-Behind

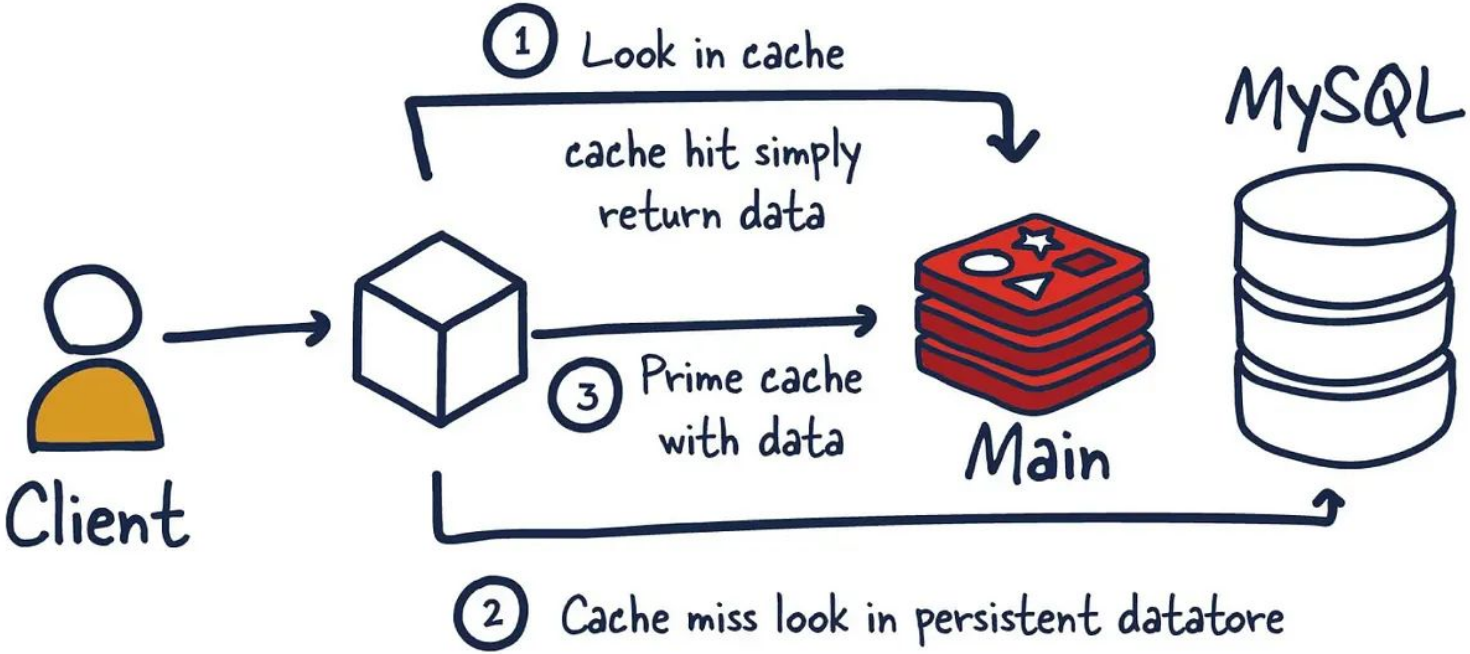
У цій стратегії дані спочатку записуються в кеш, а потім у базу даних пізніше. Це дозволяє пришвидшити операції запису, але це може призвести до неузгодженості даних, якщо кеш-пам’ять не керується належним чином.



## Read-Through

У цій стратегії кеш використовується як основне джерело даних. Коли запитуються дані, спочатку перевіряється кеш. Якщо даних немає в кеші, вони витягуються з бази даних і зберігаються в кеші для подальшого використання. Ця стратегія може бути корисною, коли база даних працює повільно або коли дані часто читаються, але рідко оновлюються.



redis

# How is redis traditionally used